

CUDA & GPU Programming Practice Test

Part 3 - Shared Memory, Atomic Ops, Warps & Kernel Mechanics — Questions 31-45 of 60 — Answers & Explanations

31. What is the purpose of practicing GPU coding problems on platforms like LeetGPU?

- A. To design new GPU hardware
- B. To practice CPU-only algorithm design exclusively
- C. To build practical, hands-on CUDA problem-solving skills through smaller, focused coding challenges
- D. To replace the need for any CUDA compiler

Correct Answer: C

Explanation: Platforms like LeetGPU provide bite-sized coding challenges focused specifically on GPU programming, helping reinforce CUDA concepts through direct practice.

32. Why is understanding both CUDA C/C++ and Python-based GPU programming (CuPy/Numba) valuable?

- A. CUDA C/C++ is always obsolete compared to Python
- B. They are functionally identical with no practical differences
- C. Different projects may favor low-level control in C/C++ or rapid development and prototyping in Python
- D. Python-based GPU programming cannot run on real hardware

Correct Answer: C

Explanation: Having both skill sets lets a developer choose low-level, fine-tuned control in C/C++ or fast, high-level development using Python-based tools, depending on the project's needs.

33. What is a common performance bottleneck that memory coalescing techniques aim to address?

- A. Too many available Streaming Multiprocessors
- B. Inefficient global memory access patterns that waste available memory bandwidth
- C. Overuse of constant memory for read-only data
- D. Excessive use of the CPU during kernel execution

Correct Answer: B

Explanation: Poorly coalesced memory access wastes memory bandwidth by requiring more transactions than necessary; coalescing techniques directly target this inefficiency.

34. Why is shared memory considered faster than global memory in a CUDA GPU?

- A. It is automatically duplicated for every thread in the kernel
- B. It has unlimited capacity compared to global memory
- C. It is located on-chip, close to the SM's compute cores, resulting in much lower latency than off-chip global memory
- D. It requires no synchronization between threads

Correct Answer: C

Explanation: Shared memory resides physically closer to the processing cores on the SM, giving it significantly lower latency and higher bandwidth than off-chip global memory.

35. What is a key reason atomic operations, while useful, can sometimes reduce kernel performance if overused?

- A. Serialized access to the same memory location by many threads can create contention and slow execution
- B. Atomic operations cannot be used with shared memory
- C. Atomic operations always run entirely on the CPU
- D. Atomic operations disable all other threads in the kernel permanently

Correct Answer: A

Explanation: Because atomics enforce serialized updates to the same memory location, heavy contention among many threads competing for the same address can become a performance bottleneck.

36. Why might a programmer study both naive and tiled/optimized versions of the same algorithm, such as matrix multiplication?

- A. Because tiled implementations cannot be compared to naive ones
- B. Because naive implementations are always faster in practice
- C. Because studying only one version is sufficient to understand GPU performance
- D. To understand the performance impact of specific optimization techniques by directly comparing them

Correct Answer: D

Explanation: Comparing naive and optimized implementations side by side makes the practical benefit of techniques like tiling and shared memory usage clear and measurable.

37. What is a practical benefit of learning to profile or reason about occupancy, memory coalescing, and shared memory together?

- A. It equips a programmer to diagnose and address multiple common sources of GPU performance bottlenecks
- B. It only applies to graphics rendering, not general-purpose computing
- C. It removes the need to ever test or benchmark code
- D. It guarantees a kernel will always run at maximum possible speed

Correct Answer: A

Explanation: Together, these concepts cover several of the most common performance limiters in GPU kernels, giving a programmer a well-rounded toolkit for optimization.

38. Why might an introductory CUDA C example, like Hello World, still be considered valuable in a comprehensive GPU programming course?

- A. It only demonstrates GPU rendering capabilities
- B. It eliminates the need to learn about kernel launches
- C. It is the most advanced GPU programming concept covered in the course
- D. It confirms the development environment and toolchain are correctly set up before tackling more complex kernels

Correct Answer: D

Explanation: A simple first example helps verify that the CUDA toolchain, drivers, and environment are working correctly before moving on to more complex GPU programming tasks.

39. What is a general takeaway about GPU performance optimization across topics like coalescing, tiling, shared memory, and occupancy?

- A. GPU performance is unaffected by how memory is accessed
- B. Effective GPU programming often involves understanding and balancing multiple interacting hardware-level factors
- C. Only one optimization technique is ever necessary for any CUDA kernel

D. Occupancy alone fully determines a kernel's real-world performance

Correct Answer: B

Explanation: Because these factors interact, strong GPU optimization typically requires considering several hardware-level aspects together rather than focusing on just one.

40. Why is it useful to practice both structured textbook-style exercises (like PMPP problems) and platform-based coding challenges (like LeetCode)?

- A. Only one of the two approaches is ever useful for learning CUDA
- B. Textbook exercises and coding challenges teach entirely unrelated skills
- C. Coding challenges replace the need to understand GPU architecture at all
- D. Textbook exercises build foundational understanding, while coding challenges reinforce practical, hands-on implementation skills

Correct Answer: D

Explanation: Combining conceptual textbook exercises with hands-on coding challenges helps reinforce both the theory and practical application of GPU programming concepts.

41. What is the purpose of the cudaMemcpy function in a typical CUDA C program?

- A. To synchronize all running kernels across multiple GPUs
- B. To transfer data between host (CPU) memory and device (GPU) memory
- C. To allocate memory exclusively on the CPU
- D. To compile a CUDA kernel into machine code

Correct Answer: B

Explanation: cudaMemcpy is used to copy data between host and device memory, or between different locations within device memory, specifying the direction of transfer.

42. What is the role of cudaMalloc in a CUDA C program?

- A. To allocate memory only on the CPU
- B. To automatically free all previously allocated GPU memory
- C. To launch a kernel with default parameters
- D. To allocate memory on the GPU device that kernels can access

Correct Answer: D

Explanation: cudaMalloc reserves a block of memory on the GPU device, which can then be used by kernels or copied to/from with cudaMemcpy.

43. What does threadIdx represent within a CUDA kernel?

- A. The total number of threads launched across all blocks
- B. The physical core number executing the thread
- C. The index of the current block within the grid
- D. The index of the current thread within its thread block

Correct Answer: D

Explanation: threadIdx gives a thread's position within its own block, which combined with blockIdx and blockDim is used to compute a unique global index.

44. What does blockDim represent in a CUDA kernel launch?

- A. The physical memory size allocated to the kernel
- B. The number of threads contained within each thread block
- C. The number of registers available per thread

D. The total number of blocks in the grid

Correct Answer: B

Explanation: blockDim specifies how many threads are organized within each block, a key part of the kernel's execution configuration.

45. Why are threads in CUDA organized into a hierarchy of grids and blocks rather than a single flat list?

- A. It provides a scalable and flexible way to map large numbers of parallel threads onto the GPU's hardware resources
- B. Because GPUs can only execute one thread at a time otherwise
- C. Because grids and blocks are purely a software convenience with no hardware relevance
- D. Because it is required for compiling C++ code

Correct Answer: A

Explanation: The grid/block hierarchy lets a kernel launch scale to very large problem sizes while mapping naturally onto the GPU's SM-based hardware organization.